

This lab session covers the basics of the Django development framework and the Git system. By the end of this session, students should have a very good understanding of how to install and configure a Django application and how to structure it. They should also have practical involvement with the Git system and how to use basic commands to version control their repository.

Pre-lab Preparation:

1. Install Python: Ensure that Python 3.7+ is installed in your system (macOS users shouldn't use their built-in Python version; they should have their own Python environment). Also, ensure to upgrade your pip command by typing '`python3 -m pip install --upgrade pip`'
2. Install Git: Ensure that you have git installed. (macOS users may install Xcode or install the commands without Xcode by typing '`xcode-select --install`' in the terminal command line).
3. Since we are using the command line for Git, you must generate SSH keys. Here is an example of how to do that:
(<https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent>).

Lab Activities:

Part 1: Django web development framework

Task 1: install Django

Step 1: Once you have Python installed and PIP upgraded on your system, you must create a virtual environment using '`python3 -m venv djangoenvv`'. This command will create an isolated Python virtual environment for your development. You can do this anywhere, but it **should not** be in your local git repository.

```
PROBLEMS OUTPUT TERMINAL ...
fridy@MacBook-Air-Fridy ~ % python3 -m venv djangoenvv
fridy@MacBook-Air-Fridy ~ %
```

Step 2: Once the virtual environment is created, you should activate it. You do this by changing the folder to the virtual environment you just created as follows:

> `cd path_to_virtual_env/djangoenvv`

macOS: > `source bin/activate`

Windows: > `Scripts\activate`

```
fridy@MacBook-Air-Fridy ~ % python3 -m venv djangoenvv
fridy@MacBook-Air-Fridy ~ % cd djangoenvv
fridy@MacBook-Air-Fridy djangoenvv % source bin/activate
```

Step 3: once activated, we can now install Django

> `pip install django`

```
fridy@MacBook-Air-Fridy ~ % python3 -m venv djangoenvv
fridy@MacBook-Air-Fridy ~ % cd djangoenvv
fridy@MacBook-Air-Fridy djangoenvv % source bin/activate
(djangoenvv) fridy@MacBook-Air-Fridy djangoenvv % pip install django
Collecting django
  Using cached django-6.0.4-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref<=3.9.1 (from django)
  Using cached asgiref-3.11.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse<=0.5.0 (from django)
  Using cached sqlparse-0.5.5-py3-none-any.whl.metadata (4.7 kB)
Using cached django-6.0.4-py3-none-any.whl (8.4 MB)
Using cached asgiref-3.11.1-py3-none-any.whl (24 kB)
Using cached sqlparse-0.5.5-py3-none-any.whl (46 kB)
Installing collected packages: sqlparse, asgiref, django
Successfully installed asgiref-3.11.1 django-6.0.4 sqlparse-0.5.5
❖ (djangoenvv) fridy@MacBook-Air-Fridy djangoenvv %
```

Task 2: create, configure, and organize a new Django project

Step 1: Navigate to the desired location at which you will create the Django web application. For example, 'DjangoProjects' in your home folder.

```
> mkdir ~/DjangoProjects
> cd ~/DjangoProjects/
```

```
fridy@MacBook-Air-Fridy ~ % python3 -m venv djangoenvv
fridy@MacBook-Air-Fridy ~ % cd djangoenvv
fridy@MacBook-Air-Fridy djangoenvv % source bin/activate
(djangoenvv) fridy@MacBook-Air-Fridy djangoenvv % pip install django
Collecting django
  Using cached django-6.0.4-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref<3.9.1 (from django)
  Using cached asgiref-3.11.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse<0.5.0 (from django)
  Using cached sqlparse-0.5.5-py3-none-any.whl.metadata (4.7 kB)
Using cached django-6.0.4-py3-none-any.whl (8.4 MB)
Using cached asgiref-3.11.1-py3-none-any.whl (24 kB)
Using cached sqlparse-0.5.5-py3-none-any.whl (46 kB)
Installing collected packages: sqlparse, asgiref, django
Successfully installed asgiref-3.11.1 django-6.0.4 sqlparse-0.5.5
(djangoenvv) fridy@MacBook-Air-Fridy djangoenvv % mkdir ~/DjangoProjects1
(djangoenvv) fridy@MacBook-Air-Fridy djangoenvv % cd ~/DjangoProjects1
(djangoenvv) fridy@MacBook-Air-Fridy DjangoProjects1 %
```

Step 2: Create your Django project as follows:

```
> Django-admin startproject libraryproject
```

```
(djangoenvv) fridy@MacBook-Air-Fridy DjangoProjects1 % django-admin startproject
libraryproject
(djangoenvv) fridy@MacBook-Air-Fridy DjangoProjects1 % cd libraryproject
```

Step 3: run the application to test that everything is working as intended:

```
> python manage.py runserver
```

```
(djangoenvv) fridy@MacBook-Air-Fridy libraryproject % python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

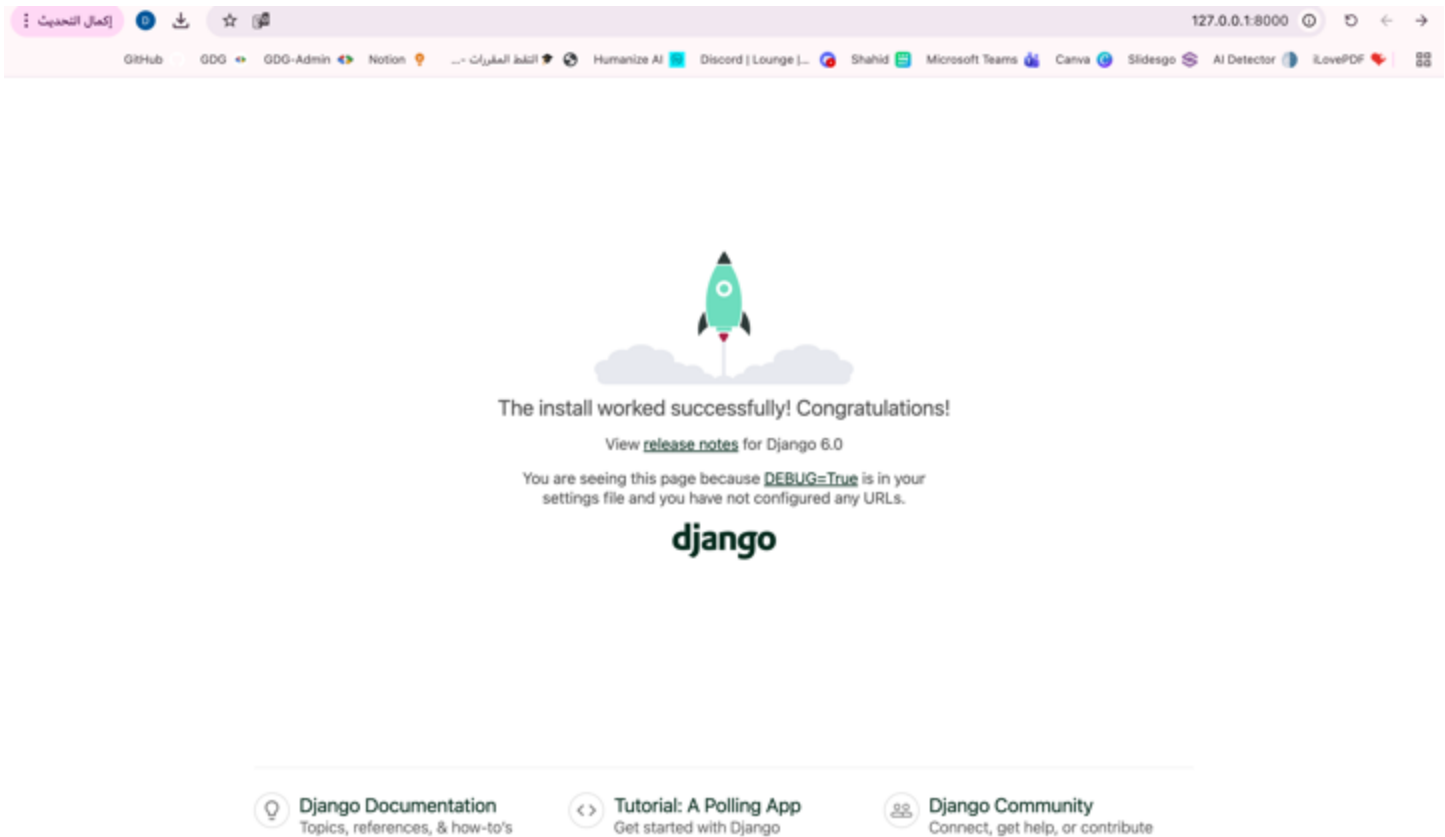
System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you
apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.

April 23, 2026 - 12:07:59
Django version 6.0.4, using settings 'libraryproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Use
a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en
/6.0/howto/deployment/
[23/Apr/2026 12:09:10] "GET / HTTP/1.1" 200 12068
[]
```

Step 4: Open your favorite internet browser and navigate to the localhost 'http://127.0.0.1:8000/'. You should see the Django default layout of the Django application.



Task 3: configure your Django application

Throughout the lab sessions, the Django application will handle two services (sub-applications). This task guides you in creating the appropriate structure for these two services. Hereafter, we will refer to the **ROOT** folder as the folder that contains the manage.py file.

Step 1: Create some folders in the **ROOT** folder

```
> mkdir apps/ apps/template/ apps/static/
(djangoenvv) fridy@MacBook-Air-Fridy libraryproject % mkdir apps apps/templates a
pps/static
```

Step 2: create two module services in the apps folder: bookmodule and usermodule

```
> cd apps
> python ../manage.py startapp bookmodule
> python ../manage.py startapp usermodule
```

These commands create two folders inside of the apps folder, and each one has a list of files as follows:

```
├── __init__.py
├── admin.py
├── apps.py
├── migrations
│   └── __init__.py
├── models.py
├── tests.py
└── views.py
```

```
(djangoenvv) fridy@MacBook-Air-Fridy libraryproject % cd apps
(djangoenvv) fridy@MacBook-Air-Fridy apps % python ../manage.py startapp bookmodu
le
(djangoenvv) fridy@MacBook-Air-Fridy apps % python ../manage.py startapp usermodu
le
(djangoenvv) fridy@MacBook-Air-Fridy apps %
```


Task 1: Create a local git repo.

Step 1: navigate to the **ROOT** folder of your Django application

```
> cd ~/DjangoProjects/libraryproject
```

Step 2: initialize your local Git repository:

```
> git init
```

(this command will create a git folder)

```
(djangoenvv) fridy@MacBook-Air-Fridy libraryproject % git init
Initialized empty Git repository in /Users/fridy/DjangoProjects1/libraryproject/.git/
```

Step 3: You don't want to add all the files from your local repository to the online one. So, create an ignore file to add all file extensions that the git system will overlook.

macOS: > touch .gitignore & open .gitignore

Windows: > type NUL > .gitignore & .gitignore

Once the file .gitignore is opened in a text editor, add the following lines:

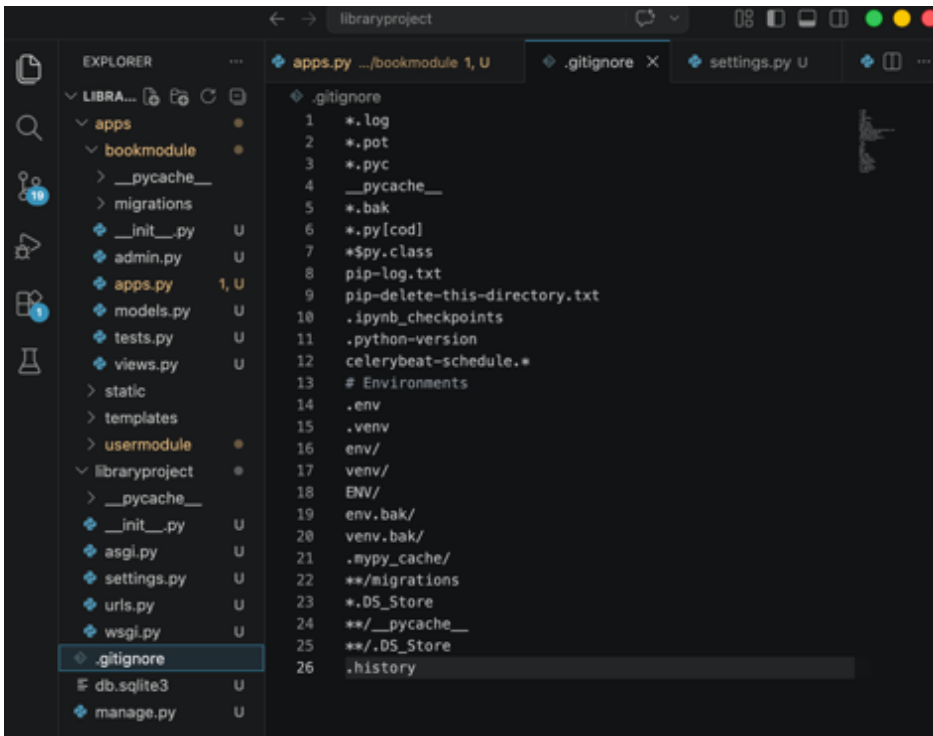
```
*.log
*.pot
*.pyc
__pycache__
*.bak
*.py[cod]
*$py.class
pip-log.txt
pip-delete-this-directory.txt
.ipynb_checkpoints
.python-version
celerybeat-schedule.*
# Environments
.env
.venv
env/
venv/
ENV/
env.bak/
venv.bak/
.mypy_cache/
**/migrations
*.DS_Store
**/__pycache__
**/.DS_Store
.history
```

Then, we need to stage the ignore file and let Git know about it.

```
> git add .gitignore
```

```
> git commit -m "add ignore file"
```

```
(djangoenvv) fridy@MacBook-Air-Fridy libraryproject % touch .gitignore
(djangoenvv) fridy@MacBook-Air-Fridy libraryproject % code .gitignore
zsh: command not found: code
(djangoenvv) fridy@MacBook-Air-Fridy libraryproject % code .gitignore
(djangoenvv) fridy@MacBook-Air-Fridy libraryproject % git add .gitignore
(djangoenvv) fridy@MacBook-Air-Fridy libraryproject % git commit -m "add ignore
[main (root-commit) 2a9eac6] add ignore file
1 file changed, 26 insertions(+)
create mode 100644 .gitignore
(djangoenvv) fridy@MacBook-Air-Fridy libraryproject %
```

**Task 2: connect your local repository with the remote repository you created in Lab 2.**

Step 1: before you can do anything, we must first configure git to use the SSH connection with your remote repository. Use the same credentials that you used to create your online repository. Then, you should do the following:

```
> git config --global user.name "Pixel"  
> git config --global user.email pixel@webmail.com
```

```
• (djangoenvv) fridy@MacBook-Air-Fridy libraryproject % git config --global user.name "DeemaAlfridy"  
• (djangoenvv) fridy@MacBook-Air-Fridy libraryproject % git config --global user.email "alfridydeema@gmail.com"
```

Step 2: Generate the authentication keys. You can visit this [link](#) for different operating systems, here we use macOS compatibility commands.

```
> ssh-keygen -t ed25519 -C pixel@webmail.com
```

You will be prompted to enter the key's location. Press the enter/return key on your keyboard to accept the default location, or you must give the full path to your SSH configuration folder.

```
> Enter a file in which to save the key (/Users/Pixel/.ssh/id_ed25519): [Press enter]
```

Then, you will be prompted to enter a passphrase. If you don't want to set it up, you can press the enter/return key anyway.


```
(djangoenv) fridy@MacBook-Air-Fridy libraryproject % ssh-keygen -t ed25519 -C "alfridydeema@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/Users/fridy/.ssh/id_ed25519):
Created directory '/Users/fridy/.ssh'.
Enter passphrase for "/Users/fridy/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /Users/fridy/.ssh/id_ed25519
Your public key has been saved in /Users/fridy/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:JeBL5RR/9QXdyJucWpU2sIBviw79pR8eEI4V3UUX+Y alfridydeema@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
|  . +oo+.==+BB|
|  . =..o.+oB==|
|  o o=ooo* .E|
|  ..o=.*|
|  ..So =|
|  . o o +|
|  o . = o|
|  . o o|
+-----[SHA256]-----
(djangoenv) fridy@MacBook-Air-Fridy libraryproject % eval "$(ssh-agent -s)"
Agent pid 17210
```

Step 3: Add the key to ssh-agent

> eval "\$(ssh-agent -s)"

```
(djangoenv) fridy@MacBook-Air-Fridy libraryproject % eval "$(ssh-agent -s)"
Agent pid 17210
(djangoenv) fridy@MacBook-Air-Fridy libraryproject % cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3Awc11ZD13WTESAAAA1BB1f7g100T6Zfubv6JE03H72hvZe2H171T/MDsECK alfridydeema@gmail.com
(djangoenv) fridy@MacBook-Air-Fridy libraryproject %
```

Step 4: Navigate to the online git repository and copy the project link you dedicated to this module.

For example, if your repository project is hosted on GitHub, then it should have this pattern:

https://github.com/<user_name>/<repo_name>.git

Step 4: Add your remote repository to your local git repository:

> git remote add origin https://github.com/<user_name>/<repo_name>.git

```
(djangoenv) fridy@MacBook-Air-Fridy libraryproject % git remote add origin https://github.com/DeemaAlfridy/lab2.git
```

Step 5: Determine which branch your repository project has. Usually, it is called main. Then to the following:

> git fetch origin

> git push -u origin main

```
(djangoenv) fridy@MacBook-Air-Fridy libraryproject % git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 10 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 413 bytes | 413.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/DeemaAlfridy/lab2.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
(djangoenv) fridy@MacBook-Air-Fridy libraryproject %
```

Step 6: Double-check your content is successfully uploaded by visiting your remote online repository.

```

PROBLEMS OUTPUT TERMINAL
fridy@MacBook-Air-Fridy libraryproject % git add .
fridy@MacBook-Air-Fridy libraryproject % git commit -m "add full project"
[master b4f763c] add full project
19 files changed, 229 insertions(+)
create mode 100644 apps/bookmodule/_init_.py
create mode 100644 apps/bookmodule/admin.py
create mode 100644 apps/bookmodule/apps.py
create mode 100644 apps/bookmodule/models.py
create mode 100644 apps/bookmodule/tests.py
create mode 100644 apps/bookmodule/views.py
create mode 100644 apps/usermodule/_init_.py
create mode 100644 apps/usermodule/admin.py
create mode 100644 apps/usermodule/apps.py
create mode 100644 apps/usermodule/models.py
create mode 100644 apps/usermodule/tests.py
create mode 100644 apps/usermodule/views.py
create mode 100644 db.sqlite3
create mode 100644 libraryproject/_init_.py
create mode 100644 libraryproject/asgi.py
create mode 100644 libraryproject/settings.py
create mode 100644 libraryproject/urls.py
create mode 100644 libraryproject/wsgi.py
create mode 100755 manage.py
fridy@MacBook-Air-Fridy libraryproject % git push
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 10 threads
Compressing objects: 100% (17/17), done.
Writing objects: 100% (18/18), 3.50 KiB | 3.50 MiB/s, done.
Total 18 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/DemaAlfridy/lab2.git
  2a9eac6..b4f763c main -> main
fridy@MacBook-Air-Fridy libraryproject %

```